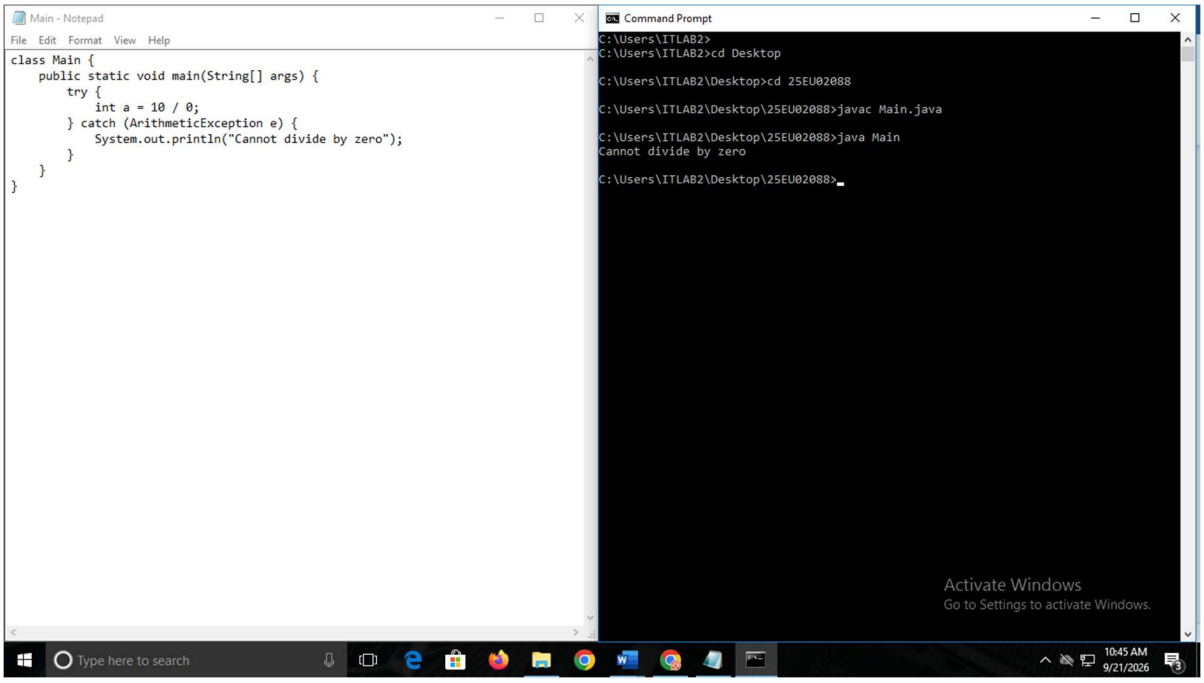
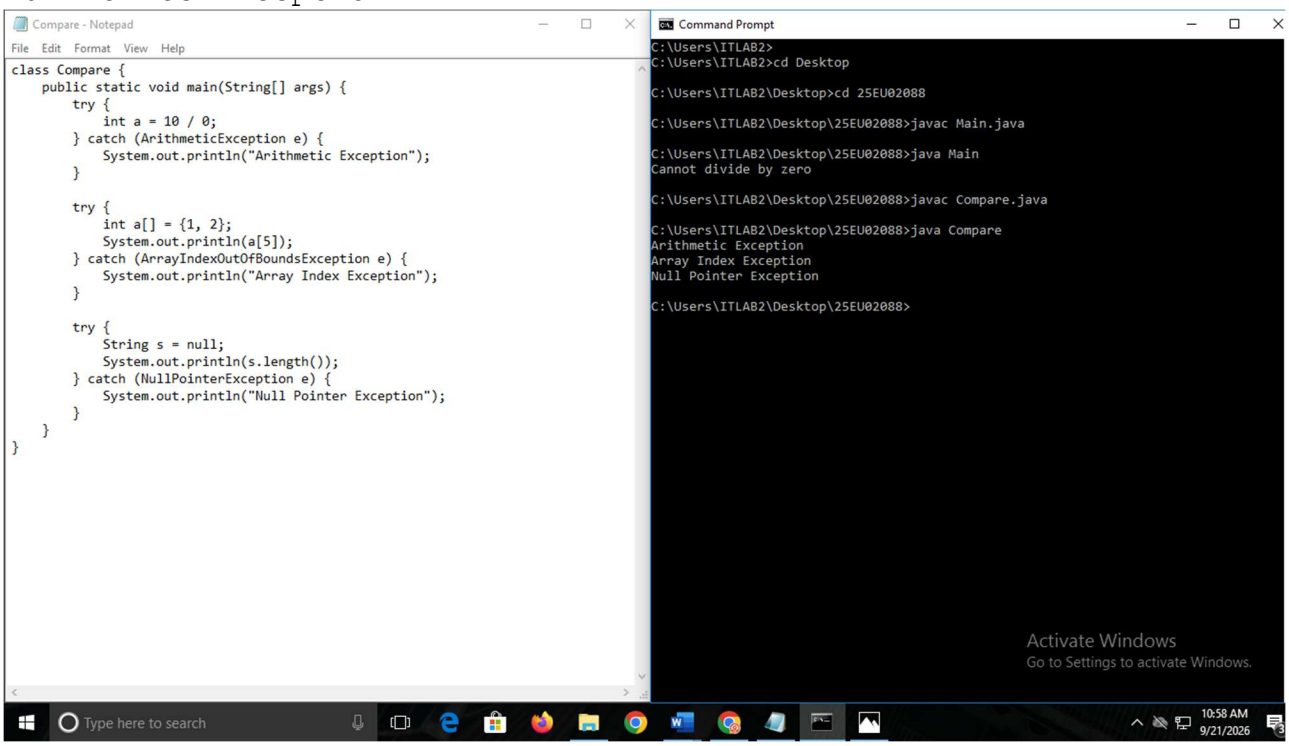
Exception Handling Experiments

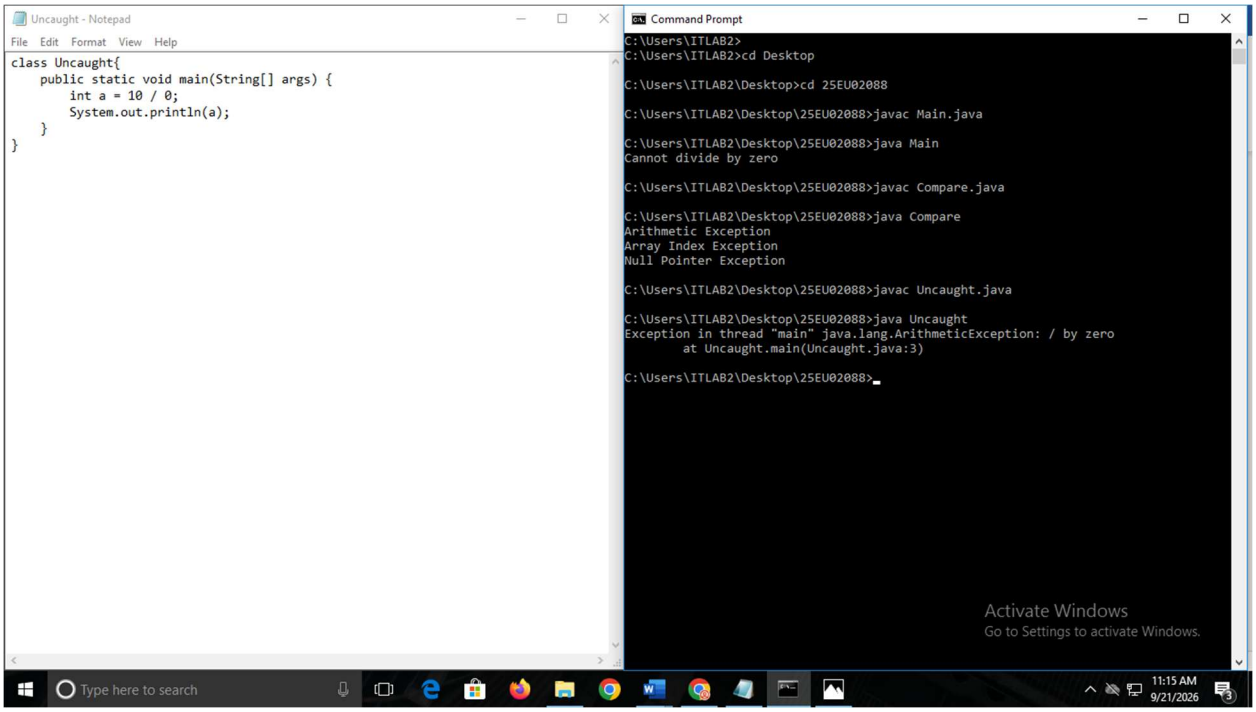
1.Write a Java program to demonstrate **exception handling using try and catch** by handling division by zero.



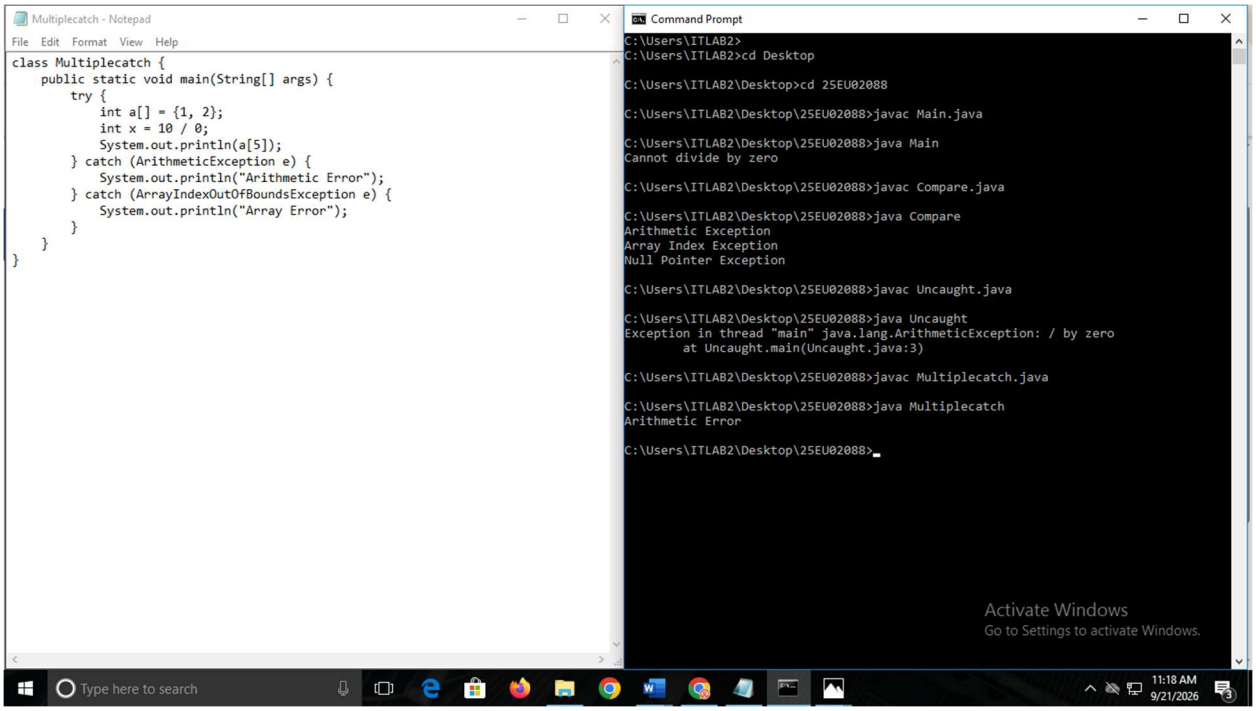
2.Write a Java program to demonstrate different **types of exceptions** such as `ArithmeticException`, `ArrayIndexOutOfBoundsException`, and `NullPointerException`.



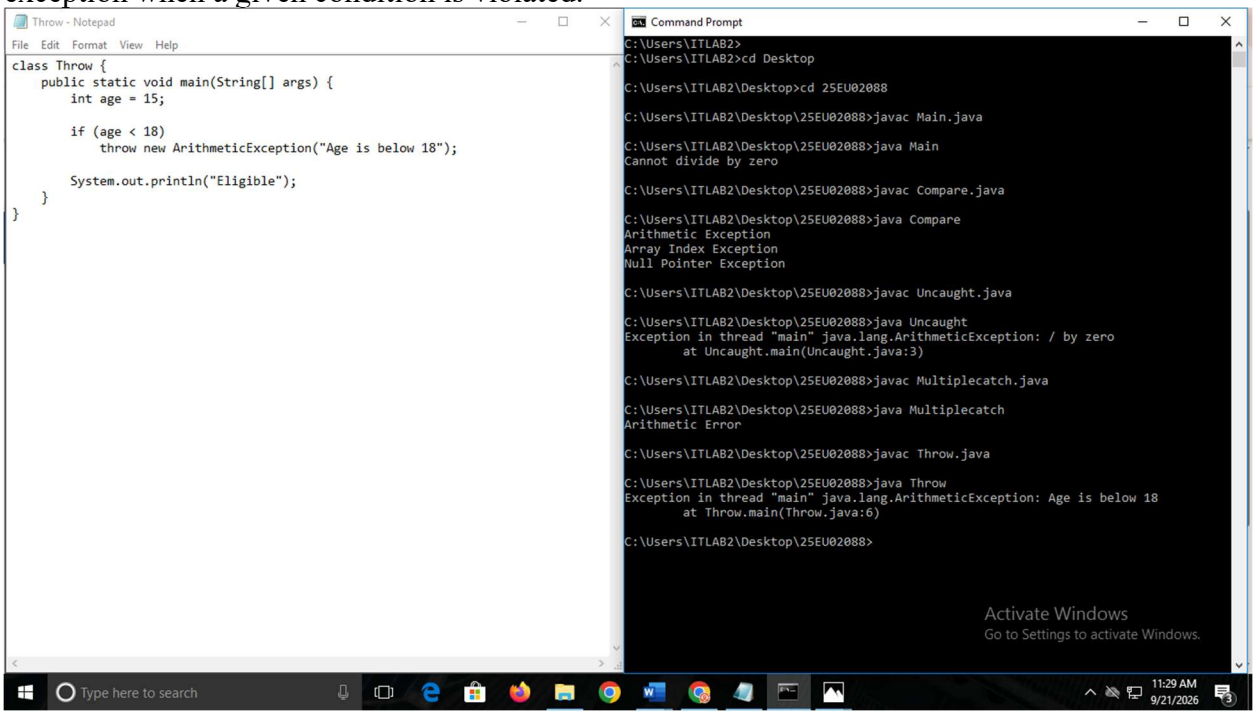
3. Write a Java program to demonstrate an **uncaught exception** and observe the default JVM exception message and stack trace.



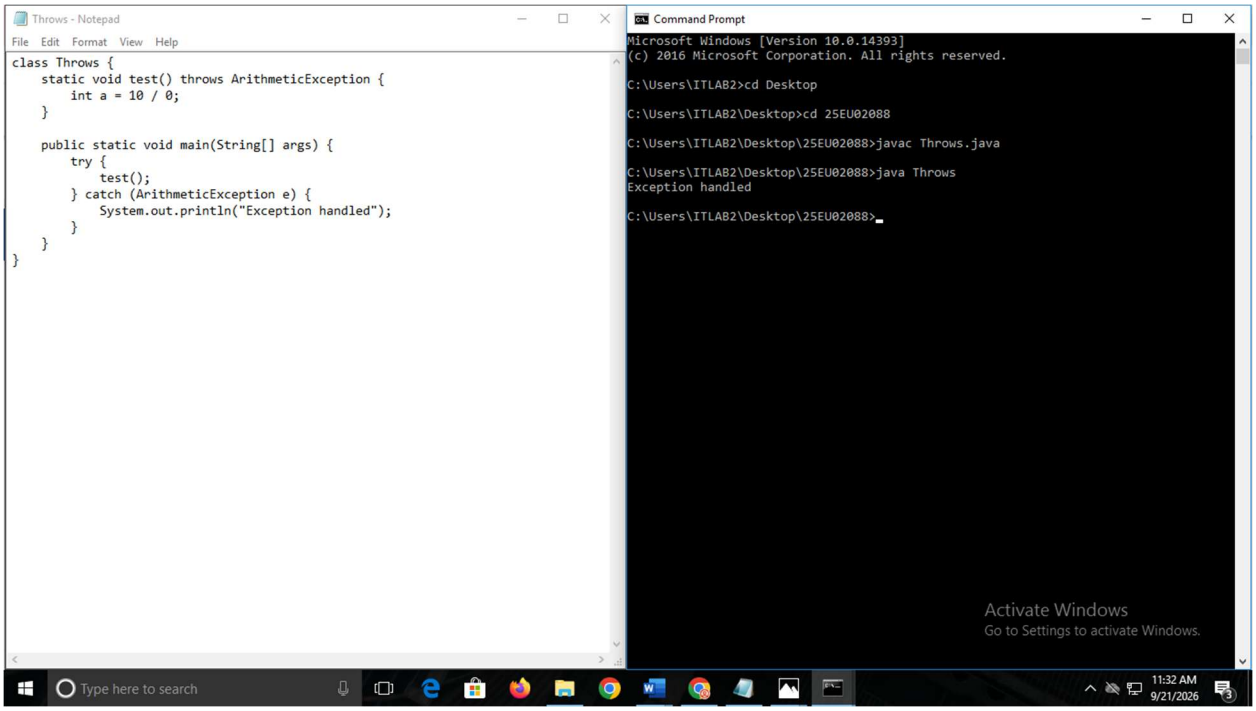
4. Write a Java program to handle **multiple exceptions using multiple catch clauses**.



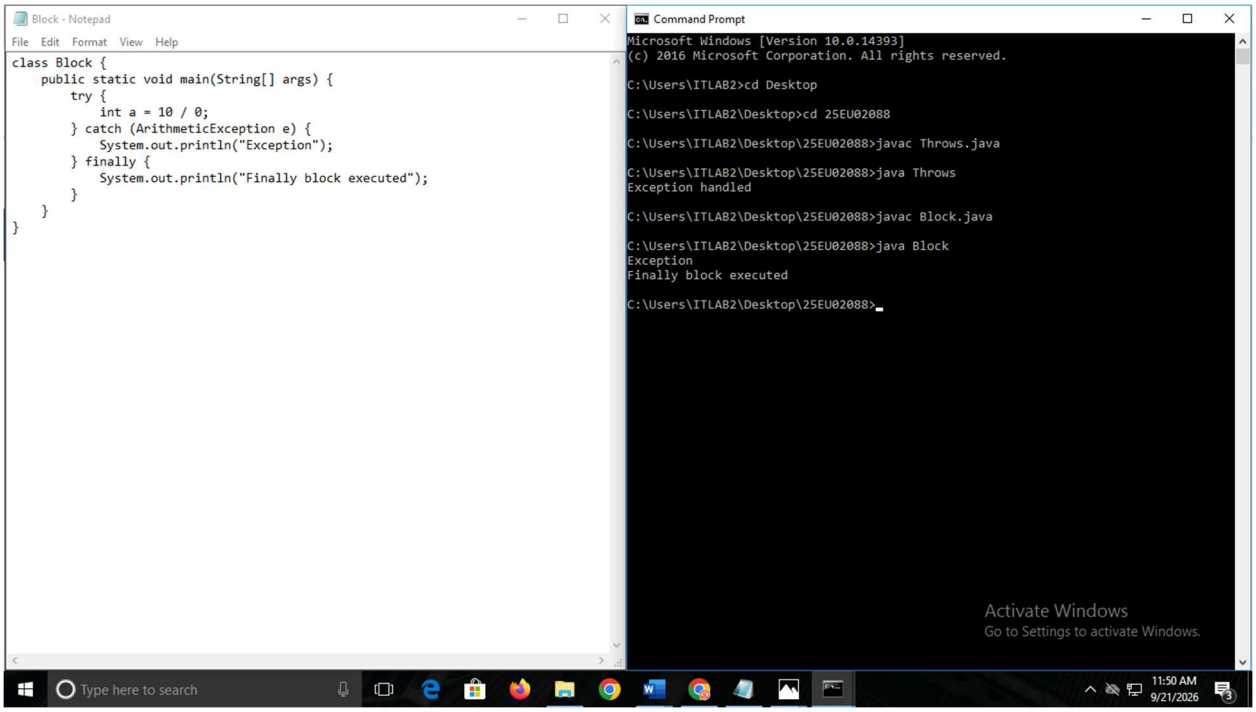
5. Write a Java program to demonstrate the use of the **throw statement** to explicitly generate an exception when a given condition is violated.



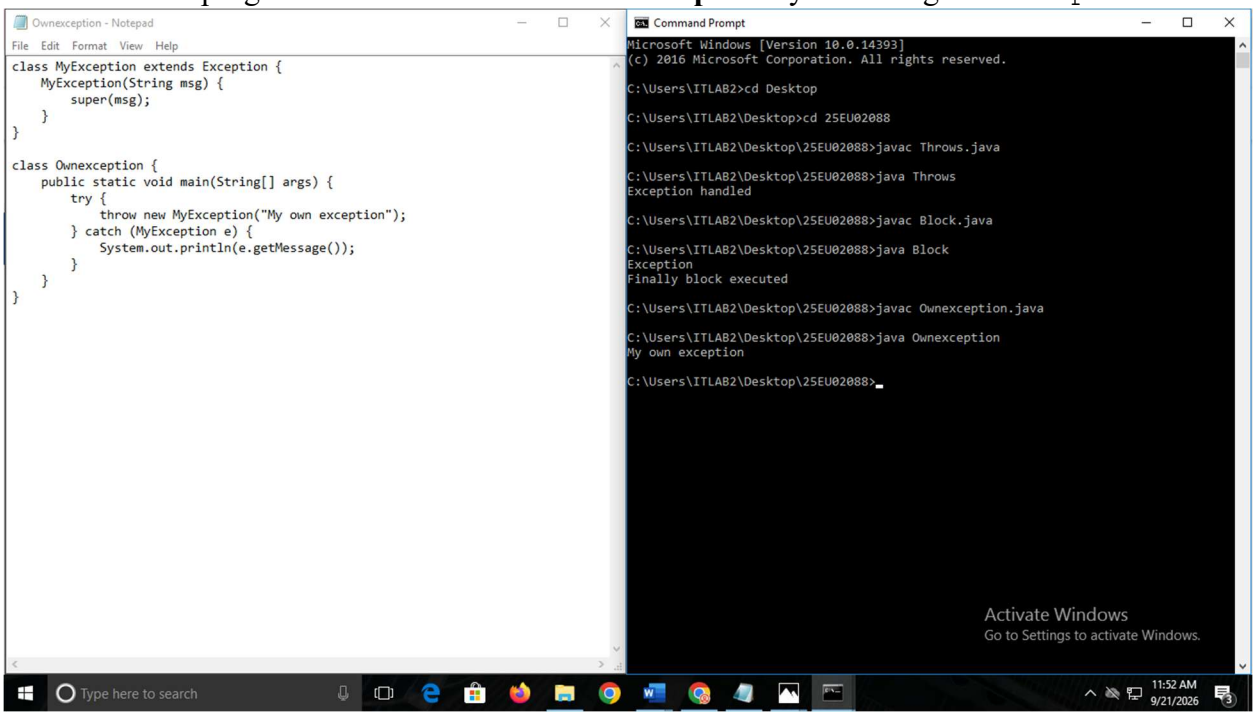
6. Write a Java program to demonstrate the use of the **throws keyword** by propagating an exception to the calling method.



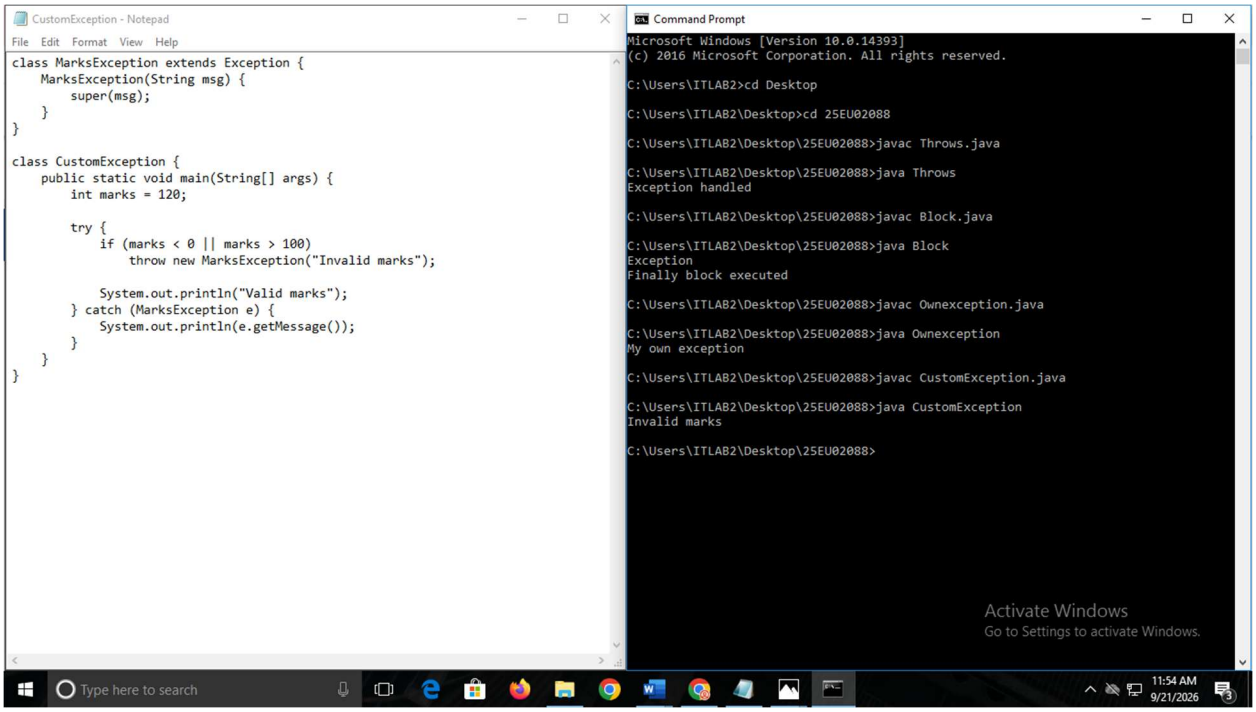
7. Write a Java program to demonstrate the use of the **finally block** for executing statements regardless of whether an exception occurs.



8. Write a Java program to create a **user-defined exception** by extending the `Exception` class.



9. Write a Java program to validate a student's marks and throw a **custom exception** when the marks are outside the valid range.



CustomException - Notepad

File Edit Format View Help

```
class MarksException extends Exception {
    MarksException(String msg) {
        super(msg);
    }
}

class CustomException {
    public static void main(String[] args) {
        int marks = 120;

        try {
            if (marks < 0 || marks > 100)
                throw new MarksException("Invalid marks");

            System.out.println("Valid marks");
        } catch (MarksException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Command Prompt

```
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\ITLAB2>cd Desktop

C:\Users\ITLAB2\Desktop>cd 25EU02088

C:\Users\ITLAB2\Desktop\25EU02088>javac Throws.java

C:\Users\ITLAB2\Desktop\25EU02088>java Throws
Exception handled

C:\Users\ITLAB2\Desktop\25EU02088>javac Block.java

C:\Users\ITLAB2\Desktop\25EU02088>java Block
Exception
Finally block executed

C:\Users\ITLAB2\Desktop\25EU02088>javac Ownexception.java

C:\Users\ITLAB2\Desktop\25EU02088>java Ownexception
My own exception

C:\Users\ITLAB2\Desktop\25EU02088>javac CustomException.java

C:\Users\ITLAB2\Desktop\25EU02088>java CustomException
Invalid marks

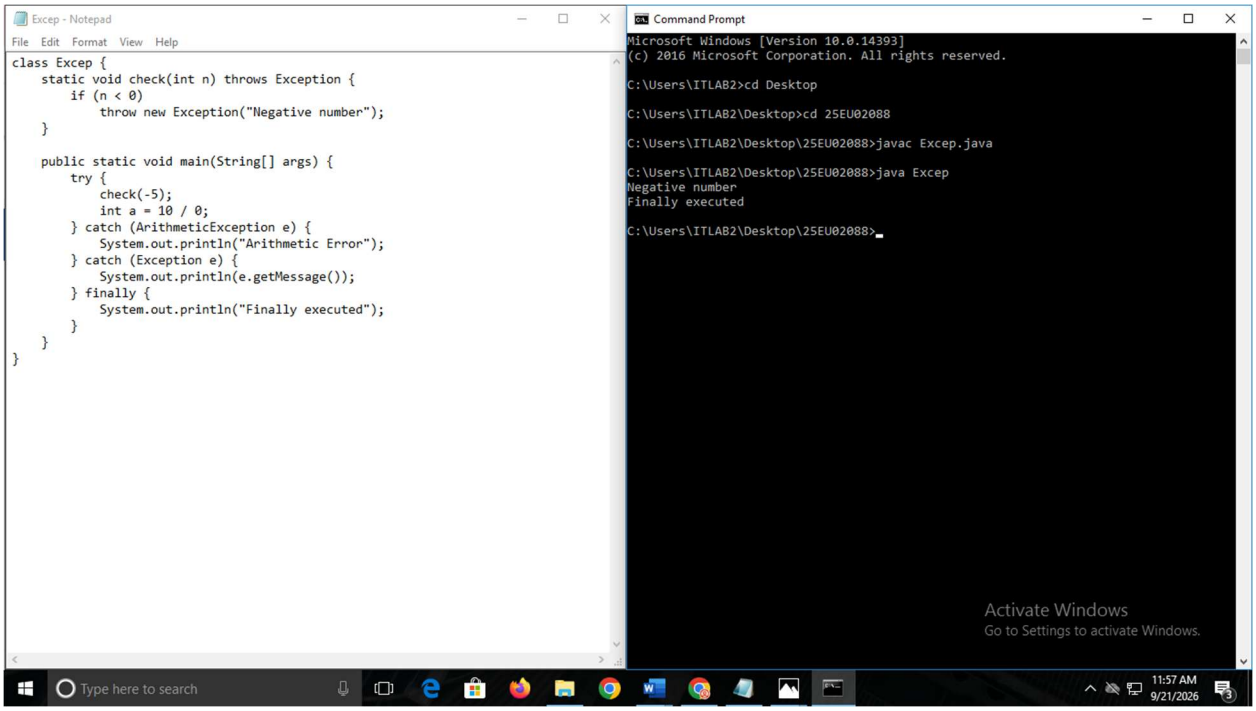
C:\Users\ITLAB2\Desktop\25EU02088>
```

Activate Windows
Go to Settings to activate Windows.

Type here to search

11:54 AM
9/21/2026

10. Write a Java program that combines **try, multiple catch, throw, throws and finally** in a single application.



Excep - Notepad

File Edit Format View Help

```
class Excep {
    static void check(int n) throws Exception {
        if (n < 0)
            throw new Exception("Negative number");
    }

    public static void main(String[] args) {
        try {
            check(-5);
            int a = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Error");
        } catch (Exception e) {
            System.out.println(e.getMessage());
        } finally {
            System.out.println("Finally executed");
        }
    }
}
```

Command Prompt

```
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\ITLAB2>cd Desktop

C:\Users\ITLAB2\Desktop>cd 25EU02088

C:\Users\ITLAB2\Desktop\25EU02088>javac Excep.java

C:\Users\ITLAB2\Desktop\25EU02088>java Excep
Negative number
Finally executed

C:\Users\ITLAB2\Desktop\25EU02088>
```

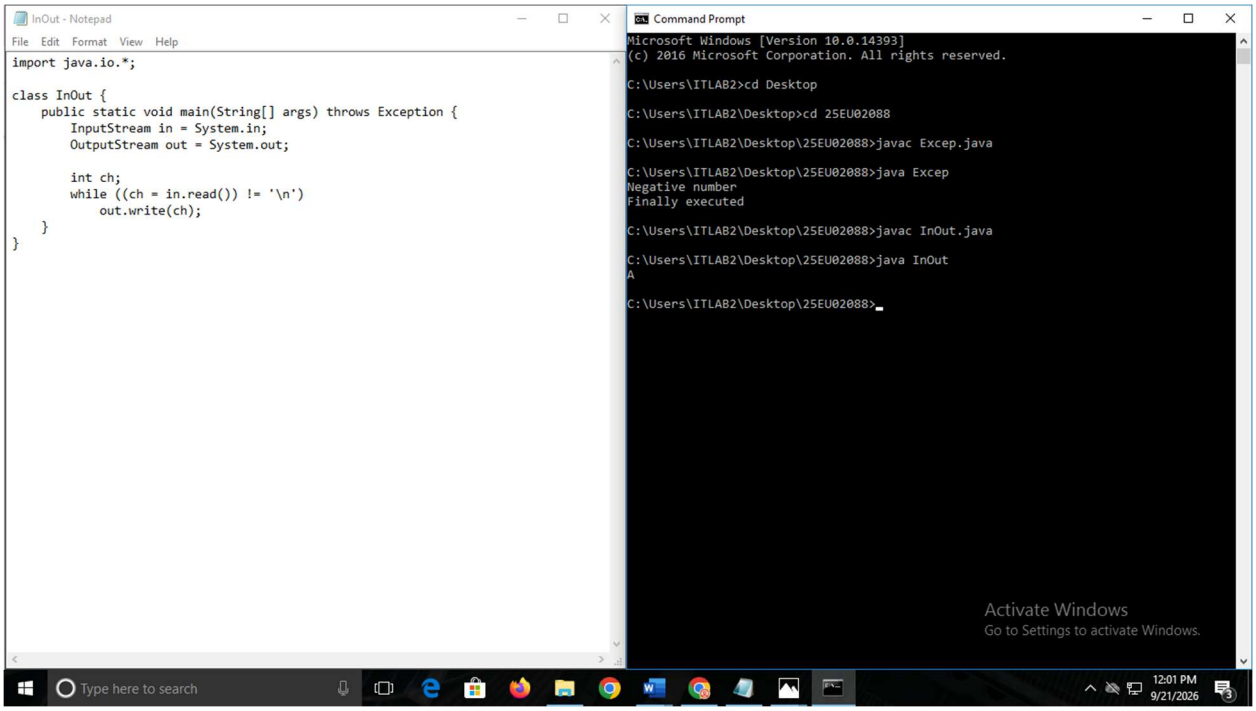
Activate Windows
Go to Settings to activate Windows.

Type here to search

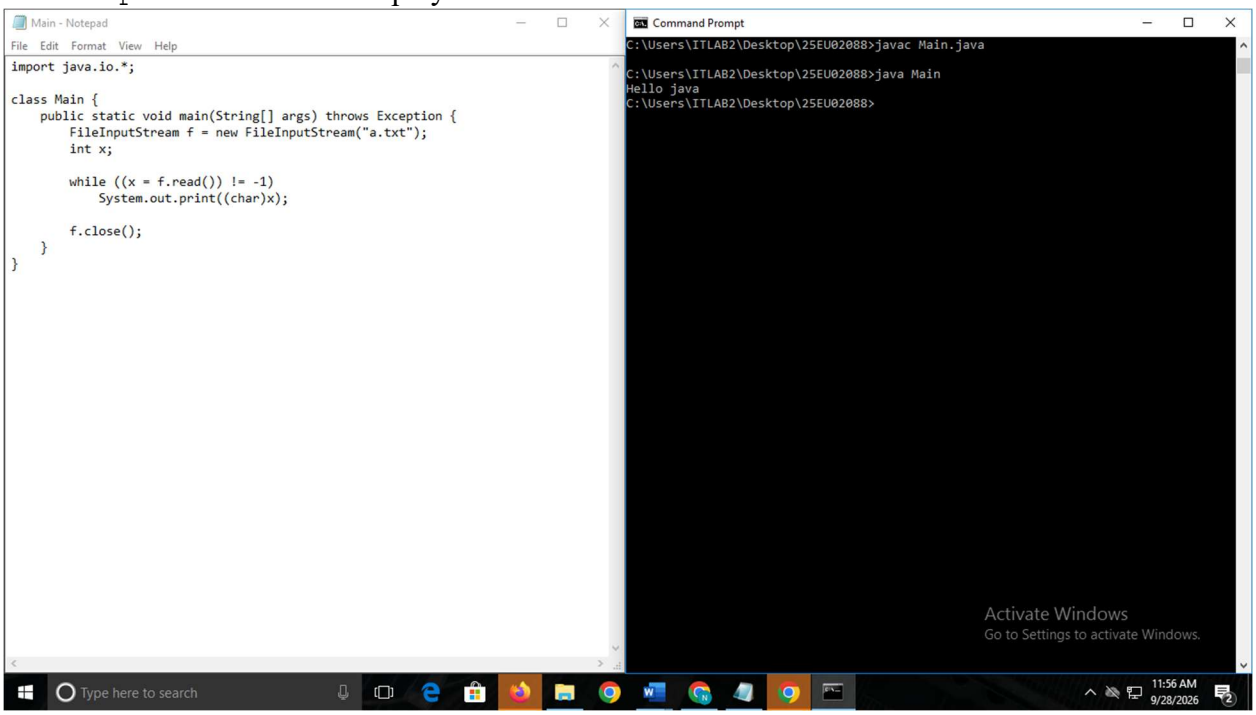
11:57 AM
9/21/2026

I/O streams: The Byte Streams Experiments

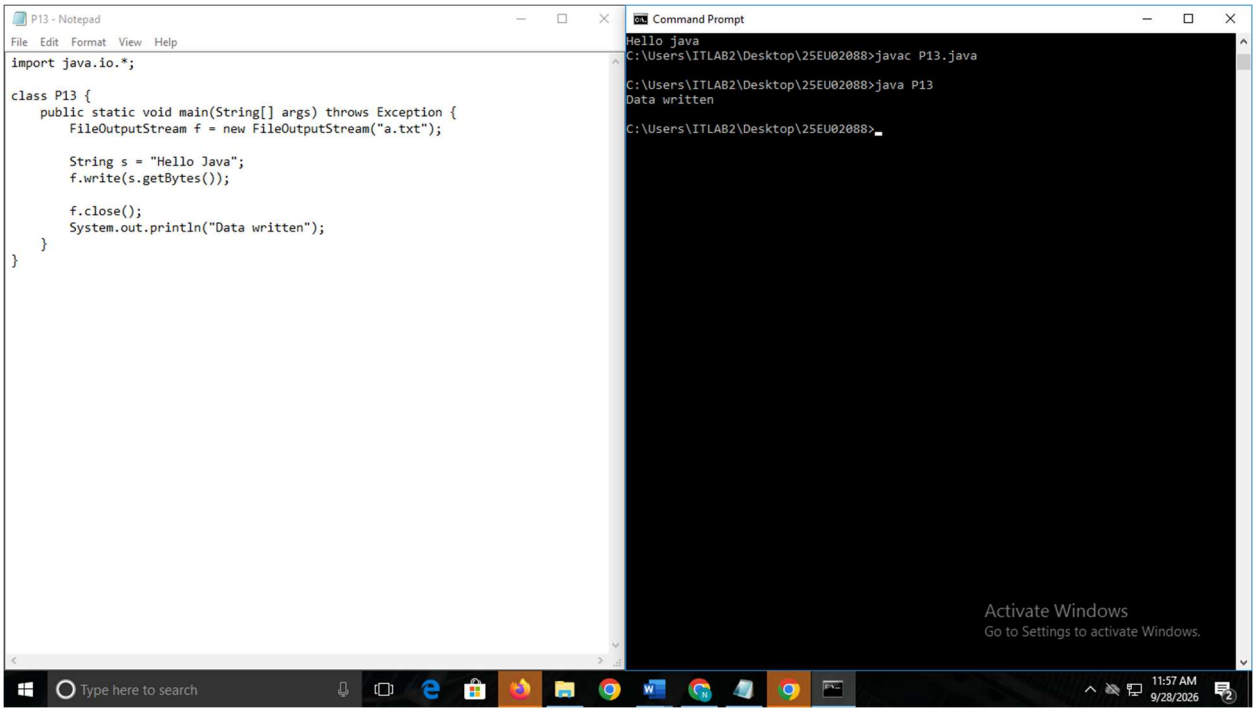
11. Write a Java program to demonstrate the use of **InputStream and OutputStream** to read bytes from an input source and write bytes to an output destination.



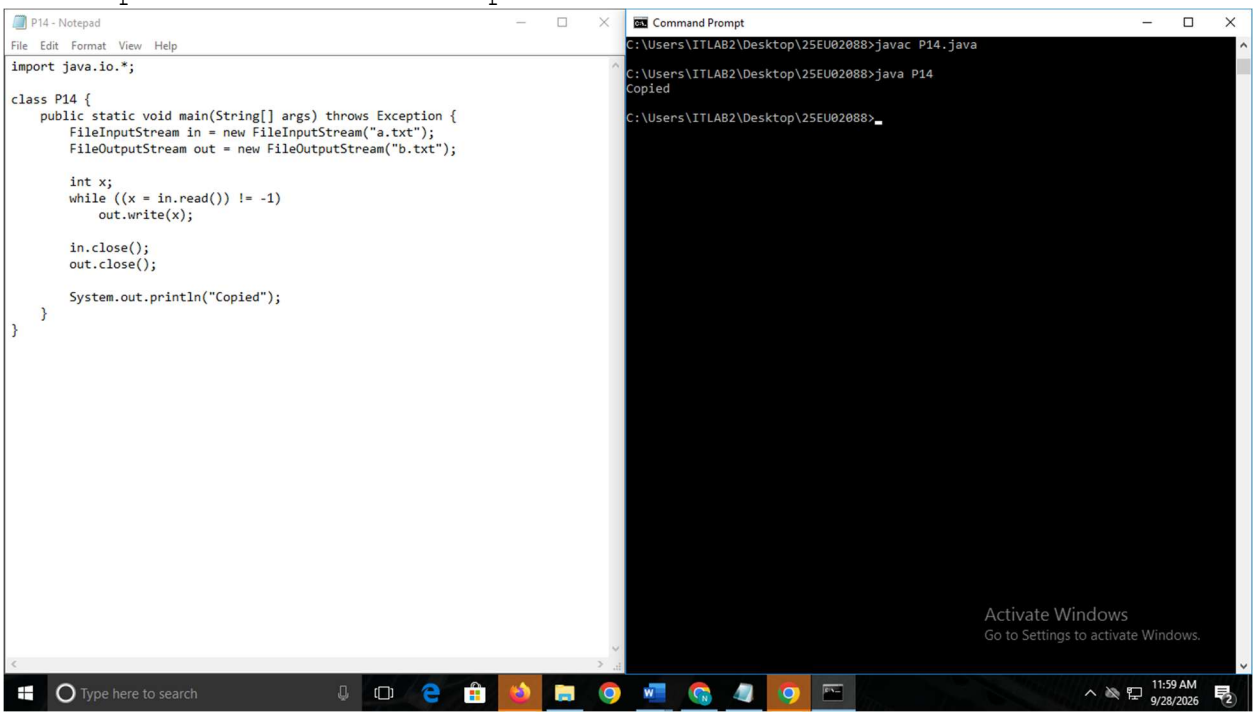
12. Write a Java program to **read the contents of a file byte-by-byte** using `FileInputStream` and display the contents on the console.



13. Write a Java program to **write data into a file** using `FileOutputStream` and verify the contents of the file.



14. Write a Java program to **copy the contents of one file to another file** using `FileInputStream` and `FileOutputStream`.



15. Write a Java program to **copy an image from one file to another** using byte streams and handle `FileNotFoundException` and `IOException` using `try-catch-finally`.

